`IConfiguration` and then configured by the `ConfigureCurrencyOptions` class.

WARNING The `CurrencyConfigureOptions` object is registered as a singleton, so it will capture any injected services of scoped or transient lifetimes.

Whether you use the `OptionsBuilder<T>` or the `IConfigureOptions<T>` approach, you need to register the `ICurrencyProvider` dependency with the DI container. In the sample code for this chapter, I created a simple `CurrencyProvider` service and registered it with the DI container using

```
builder.Services.AddSingleton<ICurrencyProvider, CurrencyProvider>();
```

As your app grows and you add extra features and services, you'll probably find yourself writing more of these simple DI registrations, where you register a `Service` that implements `IService`. The built-in ASP.NET Core DI container requires you to explicitly register each of these services manually. If you find this requirement frustrating, it may be time to look at third-party DI containers that can take care of some of the boilerplate for you.

31.3 Using a third-party dependency injection container

In this section I show you how to replace the default DI container with a third-party alternative, Lamar. Third-party containers often provide additional features compared with the built-in container, such as assembly